

TASK 1 - PERFORMANCE WITH UNLOGGED TABLES

A standard (logged) tables write every change to the WAL, and this takes more time. An UNLOGGED table skips WAL writes entirely, so inserts are faster.

insert into test_simple values (generate_series(1,1000000));

The screenshot shows a SQL IDE with the following SQL code:

```
--task2
create table labs.test_simple(a int,b int)

EXPLAIN ANALYZE
insert into labs.test_simple values (generate_series(1,1000000));
```

The results pane shows the query plan for the insert statement:

Step	Operation	Cost	Rows	Width	Actual Time	Actual Rows	Actual Loops
1	Insert on test_simple	0.00..5000.02	0	0	1346.123..1346.124	0.00	1
2	Buffers: shared hit=1008845 dirtied=4425 written=4429						
3	-> ProjectSet	0.00..5000.02	1000000	8	0.004..64.921	1000000.00	1
4	-> Result	0.00..0.01	1	0	0.001..0.002	1.00	1
5	Planning Time: 0.038 ms						
6	Execution Time: 1346.148 ms						

insert into test_simple values (generate_series(1,5000000));

The screenshot shows a SQL IDE with the following SQL code:

```
--task2
create table labs.test_simple(a int,b int)

EXPLAIN ANALYZE
insert into labs.test_simple values (generate_series(1,1000000));

EXPLAIN ANALYZE
insert into labs.test_simple values (generate_series(1,5000000));
```

The results pane shows the query plan for the insert statement:

Step	Operation	Cost	Rows	Width	Actual Time	Actual Rows	Actual Loops
1	Insert on test_simple	0.00..25000.02	0	0	6597.366..6597.366	0.00	1
2	Buffers: shared hit=5044254 dirtied=22124 written=29322						
3	-> ProjectSet	0.00..25000.02	5000000	8	0.003..307.213	5000000.00	1
4	-> Result	0.00..0.01	1	0	0.001..0.001	1.00	1
5	Planning Time: 0.022 ms						
6	Execution Time: 6597.385 ms						

insert into test_unlogged values (generate_series(1,1000000));

EXPLAIN ANALYZE INSERT INTO labs.test_unlogged VALUES (generate_series(1,1000000));	
Results 1	
EXPLAIN ANALYZE INSERT INTO labs.test_unlogged VA Enter a SQL expression to filter results (use Ctrl+Space)	
Grid	A-Z QUERY PLAN
1	Insert on test_unlogged (cost=0.00..5000.02 rows=0 width=0) (actual time=558.916..558.917 rows=0.00 loops=1)
2	Buffers: shared hit=1008845 dirtied=4425 written=4429
3	-> ProjectSet (cost=0.00..5000.02 rows=1000000 width=8) (actual time=0.003..68.807 rows=1000000.00 loops=1)
4	-> Result (cost=0.00..0.01 rows=1 width=0) (actual time=0.001..0.002 rows=1.00 loops=1)
5	Planning Time: 0.039 ms
6	Execution Time: 558.943 ms

insert into test_unlogged values (generate_series(1,5000000));

EXPLAIN ANALYZE INSERT INTO labs.test_unlogged VALUES (generate_series(1,5000000));	
Results 1	
EXPLAIN ANALYZE INSERT INTO labs.test_unlogged VA Enter a SQL expression to filter results (use Ctrl+Space)	
Grid	A-Z QUERY PLAN
1	Insert on test_unlogged (cost=0.00..25000.02 rows=0 width=0) (actual time=3312.372..3312.373 rows=0.00 loops=1)
2	Buffers: shared hit=5044254 dirtied=22124 written=41938
3	-> ProjectSet (cost=0.00..25000.02 rows=5000000 width=8) (actual time=0.004..381.750 rows=5000000.00 loops=1)
4	-> Result (cost=0.00..0.01 rows=1 width=0) (actual time=0.001..0.001 rows=1.00 loops=1)
5	Planning Time: 0.023 ms
6	Execution Time: 3312.391 ms

TASK 2– UNDERSTANDING INHERITED TABLES

Table creation and initial inserts were made

SELECT * FROM labs.users;

123 id	A-Z name	123 user_id	A-Z login
1	new Bob	1	NewLogin
999	TestUser	2	TestLogin

SELECT * FROM labs.person;

123 id	A-Z name
1	Bob
2	Alice
3	Robert
1	new Bob
999	TestUser

SELECT * FROM ONLY labs.person;

123 id	A-Z name
1	Bob
2	Alice
3	Robert

UPDATE person query results in duplicates

123 id	A-Z name
2	Alice
3	Robert
1	not Bob
999	TestUser
1	not Bob

UPDATE person ONLY query changes the primary person table Bob name

123 id	A-Z name
2	Alice
3	Robert
1	Bob on update
999	TestUser
1	not Bob

ALTER TABLE labs.person ADD COLUMN status integer DEFAULT 0;

Users table also inherits the column

123 id	A-Z name	123 user_id	A-Z login	123 status
999	TestUser	2	TestLogin	0
1	not Bob	1	NewLogin	0

ALTER TABLE labs.person ADD CONSTRAINT status CHECK (status in (0,1)) NO INHERIT;

Inserting the value with status 2 into labs.person fails

```
SQL Error [23514]: ERROR: new row for relation "person" violates
check constraint "status"
Detail: Failing row contains (100, Test, 2).
Error position:
INSERT INTO labs.person (id, name, status) VALUES (100, 'Test', 2)
```

Inserting the value with status 2 into labs.user is possible (no inherit)

```
INSERT INTO labs.person (id, name, status) VALUES (100, 'Test', 2);
INSERT INTO labs.users (id, name, status, login) VALUES (101, 'Test2', 2, 'login2')
```

users 1	person 2	person 3	Statistics 4	person 5	Statistics 6	person 7
name	Value					
Updated Rows	1					
Execute time	0.002s					
Start time	Thu Jul 09 23:42:53 EEST 2026					
Finish time	Thu Jul 09 23:42:53 EEST 2026					
Query	INSERT INTO labs.users (id, name, status, login) VALUES (101, 'Test2', 2, 'login2')					

ALTER TABLE labs.person **ADD CONSTRAINT** id **UNIQUE** (id, name);

The constraint works on person table only, the users table may bypass it (no inherit)

123 id	A-Z name	123 status
2	Alice	0
3	Robert	0
1	Bob on update	0
1	not Bob again	0
999	TestUser	0
1	not Bob	0
101	Test2	2
1	not Bob again	0

TASK 3 - BRIN VS B-TREE

The created table test_index size:

A-Z pg_size_pretty
666 MB

The execution time of the query without indexes:

27	Execution Time: 2532.332 ms
----	-----------------------------

The execution time of the query with B-Tree index:

15	Execution Time: 295.408 ms
----	----------------------------

And its size:

	AZ pg_size_pretty
1	338 MB

The execution time of the query with BRIN index:

27	Execution Time: 299.664 ms
----	----------------------------

And its size:

	AZ pg_size_pretty
1	40 kB

Conclusion:

BRIN and B-Tree gave nearly identical query speedup, but BRIN's index size was significantly smaller (40 kB vs 338 MB).

TASK 4 - GIN VS GIST

1.The size of the created table is 652 MB

2.GIST Index query time:

12	Execution Time: 1061.680 ms
----	-----------------------------

Gist index size: 1769MB

3. GIN Index query time:

12	Execution Time: 14.950 ms
----	---------------------------

Gin index size: 600MB

Conclusion:

GIN wins on both parameters: it's significantly faster than GiST (14.95 ms vs 1061.68 ms) *and* smaller (600 MB vs 1769 MB, GiST is actually bigger than the table itself).

TASK 5 - CSV FILE AS A TABLE

The materialized view on foreign table was created

Statistics 1	Statistics 2	test_foreign_table 3	test_forei
Name	Value		
Updated Rows	128		
Execute time	0.03s		
Start time	Fri Jul 10 00:45:22 EEST 2026		
Finish time	Fri Jul 10 00:45:22 EEST 2026		
Query	CREATE MATERIALIZED VIEW labs.mview_test_foreign AS SELECT * FROM labs.test_foreign_table		

The row count is 128

After the deleting rows on csv:

```
SELECT count(*) FROM labs.test_foreign_table;
```

Rows count 114

```
SELECT count(*) FROM labs.mview_test_foreign;
```

Rows count 128

After the refreshing materialized view:

```
SELECT count(*) FROM labs.mview_test_foreign;
```

Rows count 114

Conclusion:

A foreign table created has no stored data of its own — every query re-reads the file, so the row count updated to 114 immediately after the CSV was edited. A materialized view, by contrast, physically stores a snapshot of its query result at creation time — it kept showing 128 even after the file changed, because it wasn't re-querying the source. Only after refresh it re-execute its query and updates to 114..